

3.1 Actividades de reflexión inicial:

Actividad de Aprendizaje 1: El colapso del monolito sin arquitectura

- **Denominación:** Sensibilización sobre el impacto del acoplamiento, la falta de cohesión y la ausencia de diseño arquitectónico en sistemas de software escalables.
- **Duración:** 4 horas.
- **Modalidad:** Presencial / Entornos Virtuales.

A. Descripción de la Actividad

Estimado Aprendiz:

En el desarrollo profesional de software existe una dura realidad: *un código que compila, ejecuta y funciona perfectamente para 10 usuarios puede ser un fracaso rotundo y colapsar por completo cuando entran 10,000 usuarios en simultáneo*. Muchos desarrolladores novatos se enfocan exclusivamente en escribir líneas de código funcionales, ignorando por completo cómo se estructuran, comunican y aíslan los componentes del sistema. Esto se conoce como la creación de un "Monolito de Espagueti".



En esta actividad inicial nos sensibilizaremos frente a la importancia del diseño arquitectónico antes de la fase de codificación a través de dos momentos dinámicos:

1. **Momento Lúdico:** Resolverás una sopa de letras técnica para identificar los dolores de cabeza más comunes de la falta de arquitectura y los pilares para solucionarlos.
2. **Momento de Reflexión (Estudio de Caso):** Analizarás la caída técnica de una Startup en crecimiento y participarás en una mesa redonda de debate técnico.

B. Dinámica Lúdica: Sopa de Letras — Arquitectura y Deuda Técnica

Encuentra las **10 palabras** clave ocultas en la matriz, las cuales representan conceptos fundamentales del diseño y las consecuencias de omitirlo:

Monolito sin arquitectura

E	S	C	A	L	A	B	I	L	I	D	A	D	H
U	T	I	P	E	T	A	W	A	A	P	G	R	X
F	S	Y	M	I	G	R	A	C	I	O	N	F	C
C	O	L	A	P	S	O	Q	W	I	J	H	O	T
C	D	A	R	Q	U	I	T	E	C	T	U	R	A
K	A	H	N	X	S	B	W	G	C	K	U	R	P
L	A	G	Y	J	I	W	A	P	O	A	Y	H	Z
D	E	U	D	A	T	E	C	N	I	C	A	L	X
G	W	C	Y	K	I	Z	D	D	I	S	E	Ñ	O
C	O	H	E	S	I	O	N	K	S	N	Q	O	D
T	U	Q	I	N	T	E	R	F	A	C	E	S	G
M	O	N	O	L	I	T	O	R	K	A	Z	L	Z
A	X	E	G	U	N	U	E	B	O	P	F	U	N
L	H	A	C	O	P	L	A	M	I	E	N	T	O

Las palabras están ocultas horizontalmente.

Glosario de Palabras a Buscar:

1. **MONOLITO:** Sistema de software donde todos los componentes están unificados en un solo bloque ejecutable.
2. **ACOPLAMIENTO:** Grado de dependencia que existe entre dos o más módulos de software. (Buscamos que sea bajo).
3. **COHESION:** Grado de enfoque y especialización en el que un módulo realiza una sola tarea. (Buscamos que sea alta).

4. **ESCALABILIDAD:** Capacidad del software para adaptarse al aumento de carga de usuarios sin perder rendimiento.
5. **COLAPSO:** Falla total del sistema debido a la saturación de recursos por mal diseño estructural.
6. **MIGRACION:** Proceso de mover o transformar el software hacia nuevas plataformas o arquitecturas (ej. de Monolito a Microservicios).
7. **DEUDA TECNICA:** El costo futuro de elegir una solución rápida y mal diseñada en lugar de un enfoque arquitectónico correcto.
8. **DISEÑO:** Proceso de planificación y estructuración de los componentes antes de escribir código.
9. **ARQUITECTURA:** Patrones, estructuras y decisiones de alto nivel que dan forma a la solución informática.
10. **INTERFACES:** Contratos lógicos que permiten a los componentes comunicarse de forma aislada.

C. Estudio de Caso: "El Martirio de ShopQuick"

Lectura de Análisis: *ShopQuick* nació como una aplicación web pequeña de comercio electrónico. Su único programador construyó todo en un solo archivo y base de datos (Monolito): el manejo de usuarios, el inventario, los pagos y el envío de correos estaban totalmente mezclados de tal manera que las variables de la pasarela de pagos afectaban la base de datos de productos (Alto Acoplamiento).

Al principio, con 50 usuarios, el sistema volaba. El negocio fue un éxito y en el "Black Friday" pasaron de 50 a 30,000 usuarios concurrentes. Cuando el módulo de pagos se saturó por la cola de peticiones, arrastró consigo al módulo de inventario y a la autenticación. Todo el sistema experimentó un **colapso total**. Intentaron separar las tareas en servidores diferentes, pero como el código carecía de cohesión y el diseño arquitectónico era nulo, cambiar una sola línea de código rompía tres funciones diferentes. *ShopQuick* perdió miles de dólares esa semana por culpa de su arquitectura.

Preguntas Orientadoras para la Mesa Redonda:

1. A partir del caso *ShopQuick*, ¿por qué un código que inicialmente funciona bien puede convertirse en un fracaso rotundo para el negocio?
2. ¿Cómo definen en sus propias palabras la diferencia entre "*tirar código*" y "*diseñar la estructura del software*"?
3. Si modificar una parte del código obliga a reescribir otras cinco secciones del sistema, ¿estamos ante un problema de acoplamiento o de cohesión? Justifiquen su respuesta.

D. Ambiente y Materiales Requeridos

- **Ambiente:** Ambiente de formación / Aula polivalente de ADSO.
- **Materiales:** Guía de aprendizaje, Sopa de letras digital o impresa, Tablero/Videobeam para la puesta en común de la mesa redonda.

- **Material de Apoyo Externo (Alternativo a Zajuna):** Artículos técnicos abiertos sobre "*Introducción a la Deuda Técnica e Ingeniería de Software*" extraídos de portales como **InfoQ en Español** o **The New Stack**.

E. Evidencia de Aprendizaje

- **Tipo de Evidencia:** Evidencia de Conocimiento y Desempeño.
- **Entregables:**
 1. Solución visual de la sopa de letras con las palabras correctamente identificadas.
 2. Participación activa en la Mesa Redonda presencial o mediante la entrega de un documento escrito corto de 1 página con las respuestas argumentadas a las tres preguntas orientadoras del caso *ShopQuick*.